Threshold Hash-Based Signatures and Multi-Party Key Sharing

COMP6453 Project Report



The Merklers

Submission Date: Friday, 24th April 2026

Group Members
Name
Chun Kin Chau
Sai Durve
Michael Heneen
Vrudant Shah
Shep Trundle

Table of Contents

1. Introduction 3
2. Background 3
3. Problem Statement 4
4. Architectural Setup 4
   4.1 Key Generation 4
   4.2 Key Storage 5
   4.3 Key Distribution 5
   4.4 Protocols 5
      4.4.1 The Baseline Design 5
      4.4.2 The P2P Network 6
   4.5 Signing 6
      4.5.1 Baseline Signing 6
      4.5.2 Signing Policies 7
      4.5.3 Batch Signing 7
   4.6 Verification 7
5. Demo 8
6. Benchmarking Analysis 8
   6.1 Overview 8
   6.2 Lamport vs Winternitz 8
   6.3 Scalability with Number of Parties (n) 9
   6.4 XOR vs PRF Sharing 9
   6.5 Merkle Tree vs Hierarchical Merkle Tree 9
   6.6 Threshold Signing and Coalition Overhead 10
   6.7 Batch vs Individual Signing 10
7. Conclusion 10
8. Contributions 11

Appendix 14

Works Cited 17

1. Introduction

This project implements a hash-based signature (HBS) scheme that extends the traditional Lamport one-time signatures into a flexible, distributed and stateful system. We start from a baseline design which includes key generation, Merkle tree construction, share distribution and a signature verification system which we progressively optimised overtime. These include *k*-of-*n* threshold signing via coalition-based assignment, batch signing using message-level Merkle trees, optimising the Merkle tree into a hierarchical structure to support efficient key updates as well as peer-to-peer protocol. The aim of this project is to provide a practical implementation example of a scalable and efficient has-based signature scheme suitable for being extended in real-world applications.

2. Background

Early digital integrity schemes were built with non-cryptographic algorithms like checksums, which were prone to attack and tampering due to their linear structure. This ability for adversaries to tamper with information presented a dire need for secure data verification, that could not only confirm that data hasn't been changed, but also that it was sent by the correct party. Before hash-based signatures, digital signatures were built on the foundations of known hard problems such as Prime Factorization and Discrete Log. The security of these schemes is rooted in the claim that the problems themselves have no known efficient algorithm, so to break the scheme or forge a signature, an adversary must also find some algorithm that can solve the underlying problems.

In 1979, Leslie Lamport proposed a new signature scheme that was built on the assumption of a one-way hash function, using first and second preimage resistance rather than number theory assumptions. This hash-based signature scheme is known as the Lamport Signature. The Lamport Signature is unique from other schemes in that it does not create security by assuming there exists no fast solution to a problem, but instead assuming there exists no way to reverse engineer a hash function and find a message *m* from a hash *H(m)*.

In practice, the hard-problem signature schemes such as RSA and ECDSA are considered more efficient than hash-based ones like the Lamport scheme. This is because hash-based signatures require much larger signatures and keys. The Lamport scheme can use multiple kilobytes on just one signature due to the high quantity of large numbers composing a key. Compared to the tiny, non-array, signatures produced from hard-problem signatures, this an extreme disadvantage. Also, the one-time nature of the Lamport scheme means that a specific signature cannot be used more than once, and tracking algorithms and tree structures must be put in place to ensure this does not occur.

However, these disadvantages are obsolete when considering the gravity of Post Quantum Computing and the potential for disaster that lays in non-Quantum-Resistant signature schemes. The prior assumption of not being able to solve hard problems is not valid in the Post-Quantum Cryptography realm. Although the field is still relatively unknown,

attacks such as Shor's algorithm can speed up the solution on hard problems, and quantum computing is widely regarded as a threat to hard problems. But hash-based signature schemes are quantum resistant: since they only require a one-way function for security, they can simply increase the hash output to protect against brute-force attacks. While Grover's algorithm poses a threat to hash-based brute force solutions, it only provides a quadratic time increase and can be mitigated by doubling the key size. In contrast, Shor's algorithm completely breaks number theory schemes and cannot be protected against by increasing key size. Thus, as quantum computing grows into a larger and more realized threat, hash-based signature schemes such as the Lamport Signature may prove to be much more desirable when facing the task of protecting digital information.

3. Problem Statement

While hash-based signature schemes provide strong post-quantum security, they suffer from practical limitations such as large key and signature sizes, require strict one-time usage and introduce additional overhead in tracking key usage through structures like Merkle trees. Furthermore, traditional implementations assume a single signer with fill access to the private key making them unsuitable for distributed or collaborative environments.

This project addresses these limitations by implementing a distributed, stateful hash-based signature system that improves scalability, efficiency and practicality. Specifically, system extends the Lamport framework to support k-of-n threshold signing, enabling multiple parties to collaboratively generate a valid signature without exposing the full private key. The architecture also experiments with a peer-to-peer (P2P) model which removes reliance on a central authority by allowing parties to directly exchange partial signatures and collectively produce a final signature.

It additionally incorporates other optimisations such as batch signing for message-level Merkle trees and hierarchical Merkle structures to reduce signing overhead and support efficient key updates. The project also explores alternative constructions such as Winternitz signatures and PRF-based key sharing to improve storage and computation.

The goal is to demonstrate that hash-based signature schemes can be made into flexible, efficient and decentralised systems that are suitable for real-world deployment in post-quantum and distributed environments.

4. Architectural Setup

4.1 Key Generation

The Lamport private keys are composed of 256 tuples, each containing two large, randomly generated, integers. The whole private key itself is the set of all 256 tuples. The public key is the set of all 256 tuples where each value is hashed once. In Winternitz (Extension 5), this

structure is modified by replacing the pairs with hash chains, where each element is iteratively hashed multiple times to reduce signature size at the cost of additional computation.

The security of this scheme and our project as a whole is backed by the one-wayness attribute of the hash function, in this case SHA-256, which ensures that an adversary cannot compute any element of the private key based on the hashed values present in the public key. Also, the private keys are strictly one-time use, as revealing different parts of the elements provides information about the whole key.

4.2 Key Storage

After generation, the public keys are inserted as leaves into a Merkle Tree. Then the computed root of said tree is computed and made known to all parties so they can utilize it during the verification step. To improve scalability and support efficient updates, a hierarchical Merkle structure can be used (Extension 4) where leaves are grouped into subtrees and their roots are further aggregated into a higher level tree. This allows only affected subtrees to be recomputed when keys are refreshed, rather than rebuilding the entire tree.

4.3 Key Distribution

The private key corresponding to each public key is distributed among multiple parties using secret sharing to enable collaborative signing. In the baseline Lamport scheme, each private key is split into shares using XOR-based secret sharing, such that all shares combined results in the original key. To support threshold signing (Extension 1), keys are assigned to specific coalitions of size k from a total of n parties (instead of all the n parties having to sign), ensuring that only authorised subsets can produce a valid signature.

Optionally, we also include a pseudorandom function (PRF) based approach where instead of distributing explicit key material, each party is given a seed from which their share is deterministically derived during signing (while the server retains a residual share). This reduces storage overhead and improves scalability. The same approach is also extended to Winternitz (Extension 5) by deriving hash chain elements from the PRF seeds.

4.4 Protocols

4.4.1 The Baseline Design

In the baseline design, the system is structured around two distinct trust boundaries, each with its own dedicated process. The first is a trusted key server which is responsible for key generation and distribution. Once keys are generated and coalitions are assigned, the key server sends out private keys to each party, which they use to sign messages.

We also make use of a centralised untrusted server which coordinates the entire signing process. When a signing request is initiated, this server selects an available key from the Merkle tree, contacts every party in the coalition with the message, and collects their partial signatures. Each party individually computes their partial signature using their private key share and the server then combines them all (if all parties have approved). The server then retrieves the Merkle path

for the used key and creates a StatefulSignature object that can be passed to the verifier. If any party refuses to sign, the server collects all their reasons before raising an error.

While this model is straightforward and easy to understand, implement and use, it places the untrusted server in a position of power. This server selects the key, drives the protocol, collects and combines the partial signatures. This creates a single point of control that is undesirable in a distributed setting.

4.4.2 The P2P Network

Extension addresses this concern of a single server having too much power. As a solution, we restructure the protocol so that all message and signing decisions are made peer-to-peer by the individual parties. The role of the untrusted server is reduced to just a lookup service. The untrusted server now only responds to queries for helper strings such as available keys, Merkle path for given key, coalition of parties assigned to the key and public keys. The untrusted server does not know what message is being proposed for signature, who initiated the round and whether the sign was successful or not. The same trusted key server is used to generate and distribute private keys to each party.

In our proof-of –concept implementation, any party in the P2P network can initiate a signing round via its local terminal. The party initiating the signing round, queries the untrusted server for an available keyId and associated coalition members. The party then signs the message using its own private key and then broadcasts a PROPOSE message to all other members of the coalition. Each member can then accept or decline the message based on their own policies (they don't have to specifically set a policy but can approve/deny via their own terminal). The initiating party collects all responses, and like the baseline model reports all refusals before raising an error. If a unanimous acceptance is made, the initiating member combines all partial signatures to create a valid signature and then query the untrusted server for the Merkle path and to mark the key as used.

All communication in the P2P network is carried out over TCP sockets where each message is prefixed with a 4-byte length of message. We make use of python's pickle module for serialisation to transmit arbitrary objects such as key shares and Merkle paths. Party addresses and IP information is stored in a static config.json file which is read by all processes when they're started. We also make use of the trusted key server to push each party's private shares over TCP at startup and push public signing state to the untrusted server. The config.json and distribution over raw TCP is only to be used for the local demonstration; real world deployment would use a secure key distribution protocol such as TLS with mutual certificate authentication to ensure only legitimate parties receive shares.

4.5 Signing

4.5.1 Baseline Signing

In the baseline signing process, a message is signed collaboratively by a set of parties holding shares of a one-time private key. Each party independently generates a partial signature using their key share. These partial signatures are then combined using XOR to reconstruct a

valid signature. The signing server coordinates this process by validating participating parties, collecting partial signatures and producing the final combined signature. A Merkle path corresponding to the key is included to enable stateful verification.

4.5.2 Signing Policies

Each party in the system maintains a list of signing policies that determine if the message is to be signed or not. We use an abstract base class that exposes a single method called *check* which returns None or a string. It returns None if the message is okay to be signed and returns a string with the reason if the message is to be refused/violates the policy. Before a signature share is computed, we check the message across every registered policy and if the message violates a policy, we simply return a PartialSignature with success=false. This allows us to collect all partial signatures first and then combine all reasons for parties refusing to sign and raise a single error, instead of simply breaking and raising an error upon the first refusal.

The PartialSignature is itself a dataclass which carries four fields – *partyId*, *success*, *signature* and *reason*. Using this, a party can provide a structured and uniform response no matter what the outcome was. We have already set a built-in policy called - NoEmptyMessages - which refuses all empty messages. Parties can add their own new policies by subclassing SigningPolicy and implementing their own *check*. You can call the addPolicy method which enforces this contract via an isintance check ensuring that policies set are valid.

4.5.3 Batch Signing

To improve efficiency when signing multiple messages, batch signing (Extension 3) was also introduced. Instead of signing each message individually with a separate key, all messages are first aggregated into a Merkle tee and only the root of the tree is signed using a single Lamport key. Each message is accompanied by a Merkle proof that verifies its inclusion in the signed batch. This reduces the number of required signatures while preserving verifiability for individual messages.

4.6 Verification

First, the verifier must check that the given signature correlates to claimed public key. For Lamport verification, the verifier hashes each element of the signature and checks that it matches the hash stored in the public key for that specific message bit. If any one element doesn't align with the public key, then the signature is rejected. For Winternitz verification, the verifier is given a value that represents the signature element after being hashed a certain amount of time *w.* The value *w* defaults to 4 in our implementation and is also known to the verifier. The verifier then has to compute and complete the remaining amount hashes to see if it aligns with the end-of-hashing result known in the public key.

After confirming that the signature given is valid for the message, the verifier then must check a Merkle Tree containing valid public keys to ensure the public key it used is a part of the allowed set. All public keys are set up during generation and stored as leaves in the Merkle Tree. After signing, the signer additionally provides a Merkle Path for the public key it used,

and then the verifier can use that to hash its way up to the root and confirm that the leaf either belongs or doesn't belong by checking if the computed root matches the known root.

5. Demo

Please refer to the attached video for a demo of this implementation.

6. Benchmarking Analysis

6.1 Overview

This section presents a performance analysis of the implemented threshold hash-based signature system. Benchmarks were conducted across key stages of the protocol, including key generation, key distribution, signing, verification, and end-to-end execution. Additional experiments (the extensions) evaluate design choices such as sharing mechanisms (XOR vs PRF), Merkle tree structures, coalition-based threshold signing, and batch processing. All results are reported as mean ± standard deviation over repeated iterations to ensure consistency.

6.2 Lamport vs Winternitz

Referencing Graph 1, we can see a comparison between Lamport and Winternitz. This comparison reveals clear trade offs across different stages of the protocol. Key generation performance is nearly identical, with both schemes completing in approximately 1 ms, indicating that setup cost is negligible in distinguishing the two.

However, key distribution shows a significant divergence. As the number of parties increases, Winternitz becomes substantially more expensive than Lamport. At n = 10, Winternitz requires approximately 35 ms compared to 17 ms for Lamport, demonstrating roughly double the overhead. This shows that Winternitz introduces additional complexity during distribution that scales poorly with system size.

In contrast, signing performance favours Winternitz. Across all values of n, Winternitz consistently achieves faster signing times. At n = 10, Winternitz completes signing in 1.54 ms, compared to 5.16 ms for Lamport, making it over three times faster.

Despite this advantage, end-to-end results show that distribution cost dominates total performance. Lamport achieves significantly lower total execution time (23.4 ms vs 37.9 ms at n = 10), making it the more scalable option overall. This highlights a key trade-off: Winternitz optimises signing, while Lamport optimises full protocol execution.

In terms of security, Winternitz offers similar post-quantum security to Lamport but reduces signature size through hash chains. This improves storage and communication efficiency, making it preferable in bandwidth-constrained systems, despite higher computational overhead during key distribution and signing.

6.3 Scalability with Number of Parties (n)

Across all experiments, the number of participating parties (n) is the primary driver of performance. Both key distribution and signing exhibit near-linear growth as n increases, reflecting the need to perform operations per participant.

For example, Lamport key distribution increases from approximately 2.1 ms at n = 2 to 17 ms at n = 10, while signing increases from 0.6 ms to 5.2 ms. A similar trend is observed for Winternitz, though with higher distribution costs and lower signing costs.

End-to-end benchmarks further confirm that total runtime scales strongly with n, while other parameters such as Merkle tree size have comparatively minor impact. This indicates that system scalability is largely constrained by per-party computation and optimisations should prioritise reducing overhead per participant.

6.4 XOR vs PRF Sharing

Referencing Graph 2, we can observe that the choice of sharing mechanism has a significant impact on performance. XOR-based sharing consistently outperforms PRF-based approaches across both distribution and signing.

For Lamport, PRF-based signing is approximately twice as slow as XOR at small scales and remains over 2x slower at larger values of n. For Winternitz, the performance penalty is substantially greater. At n = 10, PRF-based signing requires 25.7 ms compared to only 1.54 ms using XOR, representing over a 16x slowdown.

A similar trend is observed in key distribution, where PRF introduces consistent overhead across all configurations. This indicates that PRF-based sharing incurs significant computational cost due to repeated cryptographic evaluations.

Overall, while PRF-based approaches may offer additional flexibility or security properties, XOR-based sharing is clearly the more efficient choice for performance-critical implementations.

6.5 Merkle Tree vs Hierarchical Merkle Tree

Referencing Graph 3, we observe the comparison between flat and hierarchical Merkle tree structures. This comparison highlights differences in update efficiency. Build and verification times remain relatively similar across both approaches, with only minor variations as tree size increases.

However, hierarchical Merkle trees provide clear advantages for dynamic operations. For larger trees (D = 64), hierarchical structures reduce build time and significantly improve leaf update performance. For example, leaf updates decrease from 0.436 ms in flat trees to 0.107 ms in hierarchical trees.

In contrast, verification and path generation times remain nearly identical between the two structures, indicating that read operations are unaffected by the hierarchy.

Overall, total operation time is dominated by build and update costs, meaning the improvements in hierarchical structures have a meaningful impact on end-to-end performance as tree size increases.

These results suggest that hierarchical Merkle trees are better suited for systems requiring frequent updates, as they reduce the cost of recomputing large portions of the tree.

6.6 Threshold Signing and Coalition Overhead

Reference Graph 4, Threshold signing introduces an additional combinatorial cost due to coalition generation. The number of possible signing groups grows according to C(n, k), which increases rapidly for larger configurations.

This is reflected in the benchmarking results, where coalition generation time increases from 0.002 ms for small configurations to 0.121 ms for (k = 5, n = 10), corresponding to 252 possible coalitions.

Despite this, the actual signing cost remains efficient and depends primarily on the number of participating signers (k), rather than the total number of possible combinations. For example, Lamport signing for k = 5 remains around 2.3 ms regardless of n.

This demonstrates that while coalition enumeration can introduce overhead, the signing protocol itself scales effectively with the number of active participants.

6.7 Batch vs Individual Signing

Reference Graph 5. Batch signing provides the most significant performance improvement observed in the system. By committing multiple messages into a single batch root, the cost of signing is amortised across all messages.

As batch size increases, the efficiency gain becomes more pronounced. For 16 messages, batch signing requires only 0.835 ms compared to 9.683 ms for individual signing, achieving an 11.6x speedup.

This demonstrates that batch processing effectively eliminates redundant computations, making it highly suitable for high-throughput scenarios. The results strongly indicate that batch signing is a critical optimisation for practical deployment.

7. Conclusion

In conclusion, the project demonstrates how a basic hash-based signature scheme can be extended into a more scalable system through the integration of efficiency features such as Merkle trees, threshold signing and batching techniques. We present a detailed benchmarking comparison between the baseline hash-based signature scheme features and the efficiency improvements implemented in this project. Overall, the project shows that hash-based signatures can be adapted to more realistic distributed settings while still maintaining strong cryptographic standards.

8. Contributions

Vrudant:

Implemented XOR-based additive secret sharing and threshold signing protocol where each party creates their own partial signature using its private keys and the signing server can combine these partial signatures to a full valid signature. Also added a Lamport verification function that verifies checks the full signature against a message and the public key.

I then restructured the server architecture to create the trust boundaries between a trusted KeyServer and an untrusted SigningServer. Introduced the PartialSignature dataclass as a structured return type for Party.sign().

Designed the SigningPolicy system which allows parties to reject a message based on its content. And implemented the NoEmptyMessages as the first concrete policy.

Implemented extension 2, a proof-of-concept peer-to-peer network protocol. The untrusted server is demoted to lookup service and parties communicate amongst themselves to propose and sign a message and combine signatures without a central authority. This included a TCP framing layer and an interactive CLI which allows any party to initiate a signing round from their terminal. I also made a P2PNetwork class that implements the initiator side protocol, where it queries the untrusted server for available keys and the coalition, and then broadcasts

the message to all peers. The P2PNetwork class also combines all the accepted shares into a final signature. I also implemented a PartyServer class that acts as the listening side and handles three types of incoming messages, key shares, message proposals from other parties and sign requests from the CLI. Updated the trusted KeyServer to generate key shares and then distributed them each party over a TCP socket.

I also recorded the demo presentation.

Sai:

My work on this project was mostly focused on extending the baseline Lamport hash-based signature scheme into more practical and scalable system. I implemented the initial Merkle tree integration to enable stateful signatures. Building on the base system I also designed and implemented to *k*-of-*n* threshold signing mechanism using coalition-based assignment which enhanced the distributive nature of the system. I also implemented the batch signing extension which allowed multiple messages to be aggregated into a message-level Merkle tree. Ultimately one of my main contributions to this project was enhancing the efficiency of the base pipeline.

In addition to the core features, I've also been focused on PR reviews and helping out with some other extensions such as Winternitz scheme and the hierarchical Merkle tree extension. A lot of this work involved refactoring the baseline code such that the extensions could be properly integrated in the structure.

Implementating these extensions involved being responsible for the repository structure to ensure that they could be easily integrated with the baseline code while maintaining clarity as the project became more complex. Finally, I also helped with some documentation such as development of this report.

Michael:

My contribution to this project was the design and implementation of the benchmarking and visualisation infrastructure.

I developed the benchmarking framework to measure performance across the core components of the system. This included implementing timing measurements for:

- Key generation
- Key distribution across varying numbers of parties
- Signing rounds
- Signature combination
- End-to-end protocol execution for both Lamport signature scheme and Winternitz signature scheme

To support analysis, I built a visualisation layer using Matplotlib, producing:

- Grouped bar charts for direct performance comparisons

- Heatmaps to visualise scalability and parameter sensitivity
- PRF-based key distribution and signing, compared against XOR-based sharing
- Merkle tree operations versus hierarchical Merkle tree structures, including:
    - Tree construction
    - Authentication path generation
    - Verification
    - Leaf updates
- Coalition-based threshold signing, measuring:
    - The combinatorial cost of coalition generation
    - Signing performance across different (k, n) threshold configurations
- Batch signing versus individual signing, demonstrating performance improvements when committing to a single batch root

I also wrote the analysis for Benchmarking in the report.

Poko:

Implemented Winternitz mode so the user can choose Lamport or Winternitz schemes. Adding extra function for Winternitz signature and verification. By adding extra files of Winternitz and adding extra variables input for KeySever init, it provides an extension base one the original system. (extension 4) The Winternitz file provides Winternitz's version of verification and signing, since the construction and the idea of the function are totally different. Also, changing the KeySever and SigningSever functions to fit Winternitz.

Implemented KeyGenerator for Lamport or Winternitz schemes

Implemented HierarchicalMerkle Tree (Tree of Tree) structure to store the keys. It uses Merkle trees within higher layers of the tree, whilst leaving leaves as Lamport nodes. (extension 5) It provides

- GetRoot
- MakePath
- Verify
- UpdateLeaf

Adding the test for keyGen, Winternitz and tree of tree. All of the tests cover different conditions and confirm that the basic function is safe to use.

Also provided the idea of the report and knowledge of Winternitz's performance.

Shep:

I Implemented PRF-based key generation and sharing which creates seeds and distributes them to parties separately, allowing parties to compute their key from the seed on the fly and avoiding storage of many raw keys together. In doing so I created new methods rather than change existing ones to allow the decision of raw key sharing or PRF seed sharing.

I built a new Party class, signature structure, and distribution methods to account for use of PRF seeds. Seeds are computed by server then given to parties for their own use in as a parameter in HMAC function. The PRF signature scheme can be chosen separately than standard Lamport and runs slower due to the derivation calls for each party but is much more secure.

I added specific test files to confirm working generation, signing, and verifying stages when uses PRF based key sharing with different schemes (both Lamport and Winternitz) and considering various edge cases. I also made small changes to Main to support PRF structure.

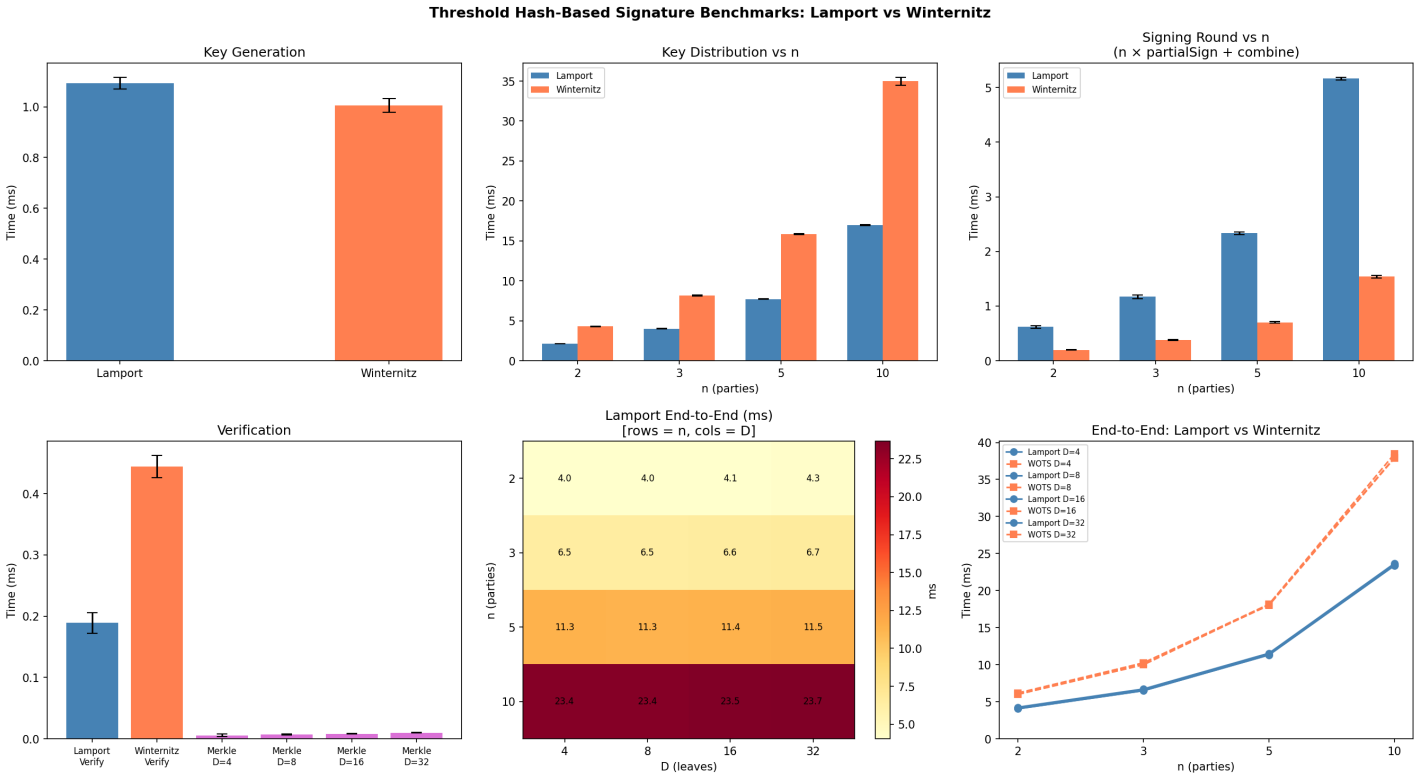
I helped develop the stateful verification scheme which improves standard LamportVerify function. This implementation allows verifier to confirm that the matching public key belongs to the set of known public keys in the Merkle Tree. The verification process creates an inclusion proof for a given leaf (public key) and walks up to the known root, countering the ability to forge a fresh key pair.

I assisted with early draft implementations of root-based verification which checks a signature using only the root of the Merkle tree and a pre-computed path rather than requiring access to the full tree. This process was later improved to use the MerkleTree class itself, with the help of Sai.

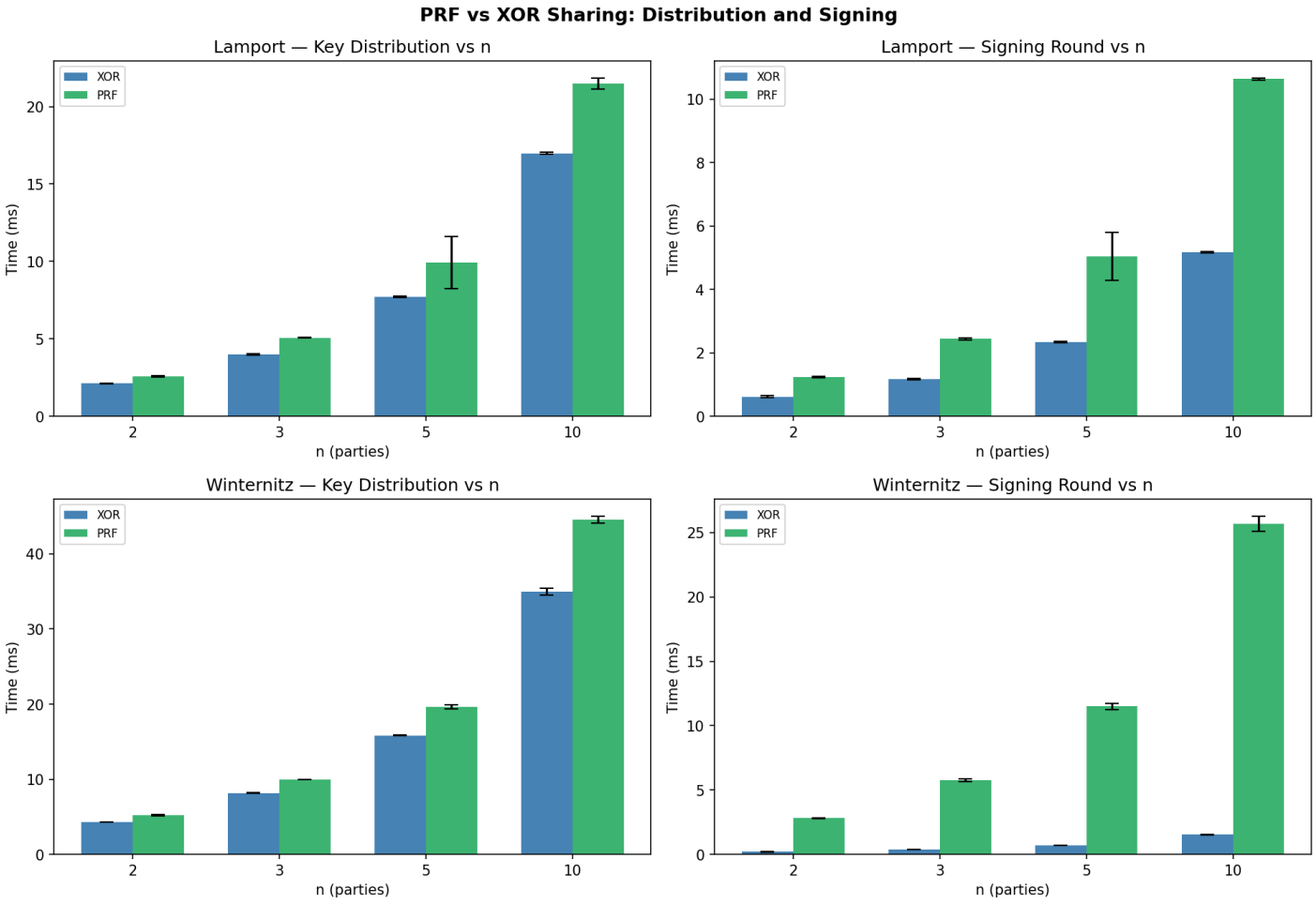
Lastly, I did background research and reporting on both the historical and modern real-world usage for Lamport signature schemes (and hash-based schemes in general) and their various design features, especially when considering Post-Quantum Computing. I also helped document our project in this report and cleanly collect our resources into a works cited.

Appendix

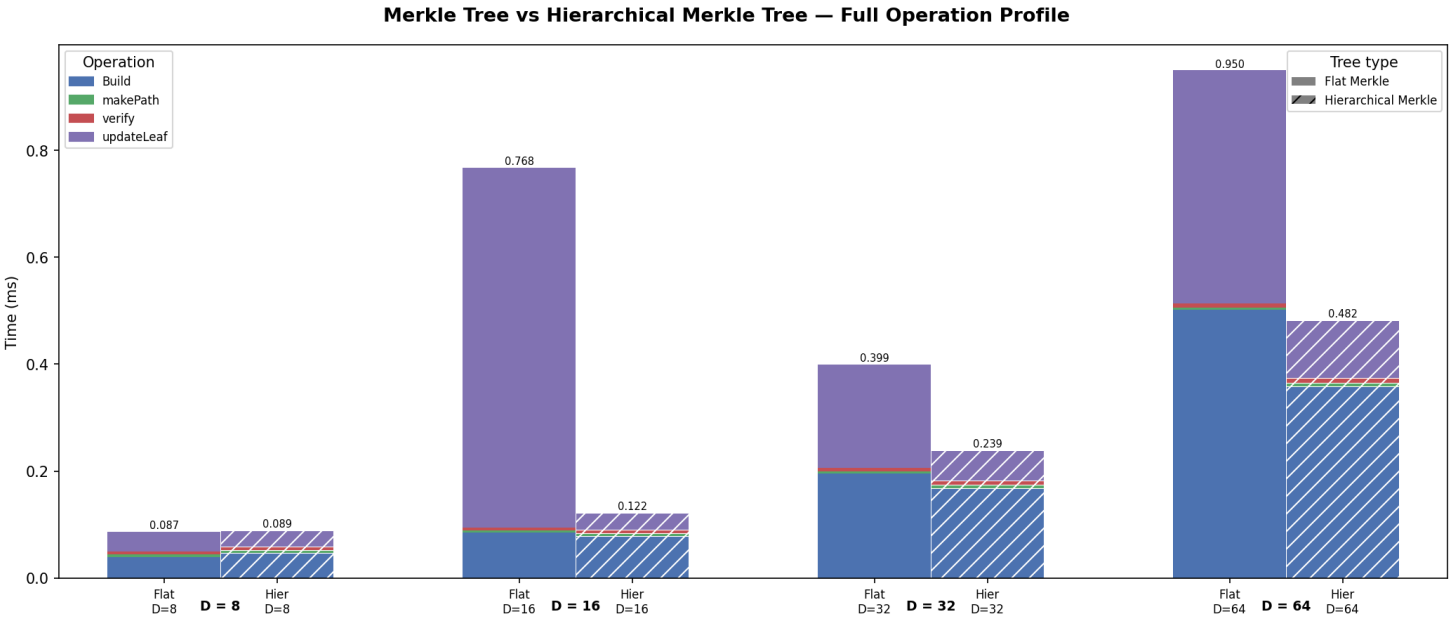
Graph 1



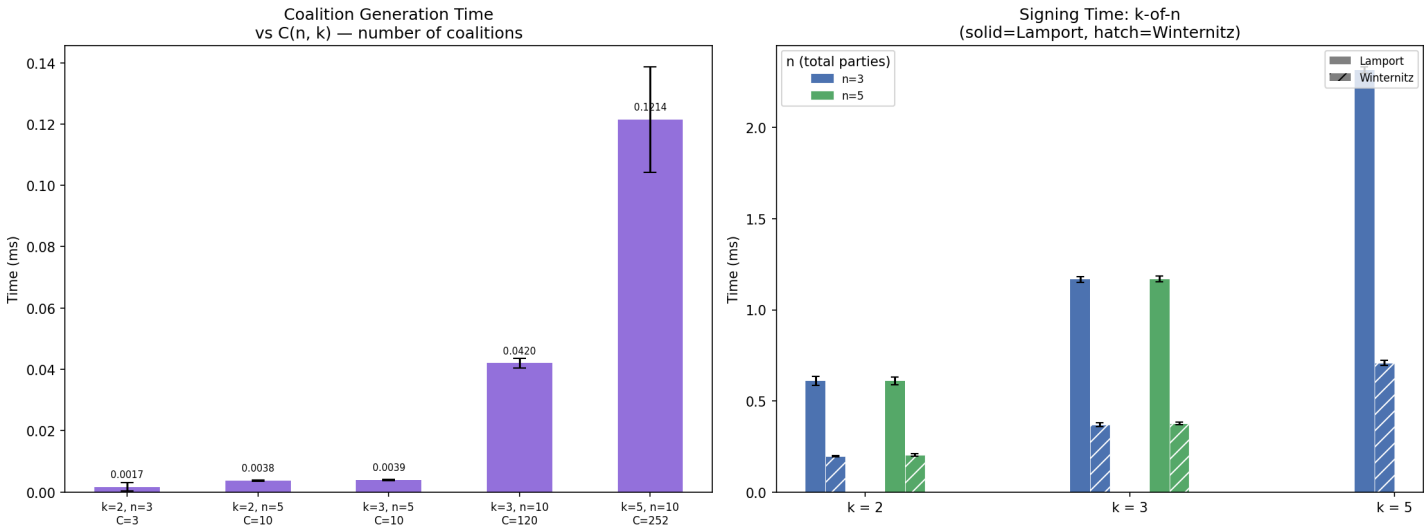
Graph 2


PRF vs XOR Sharing: Distribution and Signing
Lamport — Key Distribution vs n
Lamport — Signing Round vs n
Winternitz — Key Distribution vs n
Winternitz — Signing Round vs n
XOR
PRF
Time (ms)
n (parties)
2
3
5
10

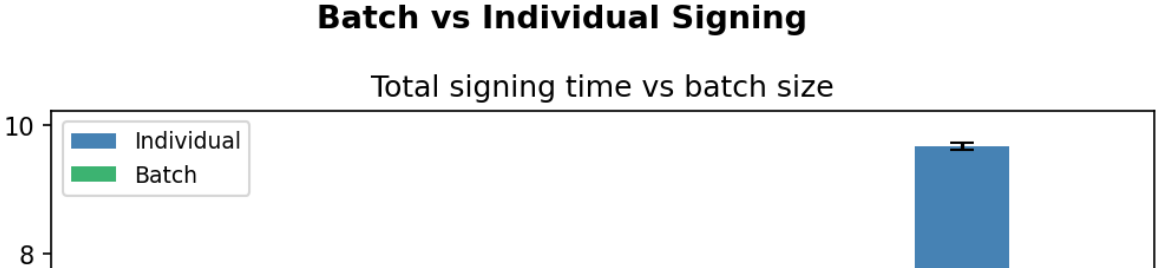
Graph 3



Graph 4



Graph 5



Works Cited

Brandão, Luís. “NIST first call for Multi-Party Threshold Schemes.” *NIST Internal Report*, Jan. 2026, https://doi.org/10.6028/nist.ir.8214c.

Green, Matthew. “Hash-Based Signatures: An Illustrated Primer.” *Cryptography Engineering*, 18 Apr. 2018, blog.cryptographyengineering.com/2018/04/07/hash-based-signatures-an-illustrated-primer/.

Kelsey, John, et al. “Haystack: Threshold and Distributed Stateful Hash-Based Signatures.” *NIST Threshold Call*, 21 Jan. 2026.

Kelsey, John, et al. “Turning hash-based signatures into distributed signatures and threshold signatures.” *IACR Communications in Cryptology*, vol. 2, no. 2, 7 July 2025, https://doi.org/10.62056/a6ksudy6b.

McMillan, Gordon. “Socket Programming HOWTO.” *Python Documentation*, 23 Apr. 2026, docs.python.org/3/howto/sockets.html#using-a-socket.

Pandit, Jay. “Hash-based Cryptography: Merkle Trees, One-time Keys, and the Quest for Quantum-safe Signatures.” *Medium*, 12 May 2025, medium.com/quantum-computing-and-cybersecurity/hash-based-cryptography-merkle-trees-one-time-keys-and-the-quest-for-quantum-safe-signatures-9572f1d88501.

Relyea, Robert. “Post-Quantum Cryptography: Hash-Based Signatures.” *Red Hat*, 27 Oct. 2022, www.redhat.com/en/blog/post-quantum-cryptography-hash-based-signatures.

“Select — Waiting for I/O Completion.” *Python Documentation*, 23 Apr. 2026, docs.python.org/3/library/select.html.

Srivastava, Vikas, et al. “An Overview of Hash Based Signatures.” *IACR Cryptology ePrint Archive*, 30 Apr. 2023, eprint.iacr.org/2023/411.

“Struct.Pack() in Python.” *GeeksforGeeks*, GeeksforGeeks, 23 July 2025, www.geeksforgeeks.org/python/struct-pack-in-python/.

“Threading — Thread-Based Parallelism.” *Python Documentation*, 23 Apr. 2026, docs.python.org/3/library/threading.html.

“Understanding Python Pickling with Example.” *GeeksforGeeks*, GeeksforGeeks, 13 Jan. 2026, www.geeksforgeeks.org/python/understanding-python-pickling-example/.